

daily-sweep-report

Technical Guide & GCP Hosting Reference

How the daily Linear sweep summary email works,
how it uses GCP Secret Manager, and how it will
be scheduled and hosted on Google Cloud.

Repository:	andrew-platformeq/tools-gcp
GCP project:	peq-tools
Job folder:	src/jobs/daily-sweep-report/
CLI command:	tools-daily-sweep-report
Generated:	June 2026

Table of Contents

1	Executive Summary
2	What the Job Does
3	Repository Layout & Module Reference
4	End-to-End Execution Flow
5	Date & Timezone Logic
6	Linear Integration
7	Issue Matching & Report Statuses
8	HTML Email Report
9	Gmail Delivery
10	Configuration & Secrets
11	GCP Integration (Detailed)
12	Hosting on GCP: Current vs Target
13	Cloud Run Job + Scheduler Architecture
14	IAM, Identity & Security Model
15	Local Development & Testing
16	Migration from GitHub Actions
17	Operations, Monitoring & Troubleshooting
18	Future Jobs in tools-gcp

1. Executive Summary

daily-sweep-report is a batch job in the tools-gcp repository. Every weekday morning it emails a summary of four recurring Linear sweep tasks for Team Mekhael: Beginning of Day and End of Day sweeps for Marco Durante and Amer Roufail.

The job replaces an earlier GitHub Actions workflow that ran inline Python with credentials stored in GitHub Secrets. The new design stores secrets in GCP Secret Manager, runs as a CLI command (tools-daily-sweep-report), and is designed to be hosted on GCP via Cloud Run Jobs triggered by Cloud Scheduler.

Key facts

Trigger (target)	Mon-Fri ~9:00 AM America/New_York
Report window	Previous business day (Fri on Monday)
Data source	Linear GraphQL API
Output	HTML email via Gmail SMTP
Secret container	peq-tools-daily-sweep-report-config
Runtime identity (GCP)	tools-gcp-run@peq-tools.iam.gserviceaccount.com
CLI entry point	tools-daily-sweep-report

2. What the Job Does

Operations teams use recurring Linear issues titled 'Beginning of Day Sweep' and 'End of Day Sweep'. Each team member creates and completes these issues daily. Managers need a single email showing whether each expected sweep was created, completed, or missing for the prior day.

Watched issue slots (4 total)

- Beginning of Day Sweep -- Marco Durante
- End of Day Sweep -- Marco Durante
- Beginning of Day Sweep -- Amer Roufail
- End of Day Sweep -- Amer Roufail

Note: Titles and assignee names must match Linear exactly (case-insensitive match at runtime).

Three possible states per slot

Done	Issue exists and state.type == 'completed'
Pending	Issue exists but not completed
Missing	No matching issue was created that day

3. Repository Layout & Module Reference

tools-gcp is structured for many jobs. Shared platform code lives in `src/tools/`. Each job gets its own folder under `src/jobs/` with kebab-case naming.

src/	
tools/	Shared: config, Secret Manager, FastAPI app
jobs/	
README.md	Conventions for adding future jobs
daily-sweep-report/	This job (kebab-case folder)
config.py	Watched issues, env settings, secret schema
secrets.py	Load JSON credentials from Secret Manager
dates.py	Report date window (America/New_York)
linear.py	GraphQL fetch + issue matching
report.py	HTML email builder
gmail.py	SMTP send via Gmail
main.py	CLI entry point (run orchestration)

Module responsibilities

config.py	WATCHED_ISSUES list; JobSettings from env; ReportSecrets schema
secrets.py	Calls tools.secrets.get_secret(); parses JSON
dates.py	yesterday_range_utc() with Mon->Fri rule
linear.py	POST to api.linear.app/graphql; match_issues()
report.py	build_email() -- HTML template + summary stats
gmail.py	send_report_email() via smtp.gmail.com:465
main.py	run() pipeline; argparse --dry-run

Python imports use `daily_sweep_report` (snake_case) mapped to the kebab-case folder via `pyproject.toml` [tool.setuptools.package-dir]. Future jobs follow the same pattern.

4. End-to-End Execution Flow

```

START (tools-daily-sweep-report)
  |
  v
Load JobSettings from env (TOOLS_* vars)
  |
  v
Load ReportSecrets from Secret Manager (JSON)
  |-- skip if TOOLS_SKIP_GCP=1 (tests / dry-run)
  v
Compute report date window (dates.yesterday_range_utc)
  |
  v
Fetch Linear issues created in that window (linear.fetch_issues)
  |-- filter: title in [BOD Sweep, EOD Sweep]
  v
Match 4 watched slots to fetched nodes (linear.match_issues)
  |
  v
Build HTML + subject (report.build_email)
  |
  v
If dry_run or skip_gcp --> log and exit 0
  |
  v
Send email via Gmail SMTP (gmail.send_report_email)
  |
  v
END (exit 0)

```

Step-by-step detail

1. `main.py` parses `--dry-run` and configures logging.
2. `load_report_secrets()` reads `peq-tools-daily-sweep-report-config` from Secret Manager using Application Default Credentials (ADC). The secret value is JSON with `gmail_address`, `gmail_app_password`, `send_to`, and `linear_api_key`.
3. `yesterday_range_utc()` determines which calendar day the report covers and returns UTC ISO timestamps for the Linear `createdAt` filter.
4. `fetch_issues()` queries Linear for all sweep-titled issues created in that window.
5. `match_issues()` pairs each of the 4 watched slots with at most one Linear issue by title + assignee name.
6. `build_email()` renders the HTML report with per-person sections, status badges, notes (description minus checklist lines), and summary counts.
7. `send_report_email()` connects to Gmail on port 465 (SSL), authenticates with the app password, and sends to `send_to`.

5. Date & Timezone Logic

All report dates use `ZoneInfo('America/New_York')`. This handles EST/EDT automatically unlike a fixed UTC-5 offset.

Which day does the report cover?

Tuesday run	Monday's sweeps
Wednesday run	Tuesday's sweeps
Monday run	Friday's sweeps (3 days back, skips weekend)

The job runs on weekdays and reports on the previous business day. On Monday, `days_back = 3` so the report covers

Friday rather than Sunday.

UTC conversion for Linear

Linear filters use createdAt with gte/lte in UTC ISO format (e.g. 2026-06-12T04:00:00Z). The local day 00:00:00-23:59:59 America/New_York is converted to UTC before the API call.

```
# Example: report for Friday June 12, 2026 (ET)
start_utc = "2026-06-12T04:00:00Z"  # midnight ET in summer (EDT)
end_utc   = "2026-06-13T03:59:59Z"  # end of day ET
```

6. Linear Integration

The job uses Linear's GraphQL API at <https://api.linear.app/graphql>. Authentication is the API key passed in the Authorization header (no Bearer prefix).

GraphQL query

Issues are fetched with a filter on createdAt (gte/lte) and title (in list). Up to 50 nodes are returned. Fields used: identifier, title, description, completedAt, assignee.name, state.type, url.

```
filter: {
  createdAt: { gte: "<start_utc>", lte: "<end_utc>" },
  title: { in: ["Beginning of Day Sweep", "End of Day Sweep"] }
}
```

Why createdAt, not completedAt?

The report tracks whether sweeps were created for the day and whether they were marked Done. Missing means no issue was created at all. Pending means created but state.type != completed. Done uses completedAt for the timestamp display.

Error handling

- HTTP errors raise RuntimeError with response body
- GraphQL errors array in response raises RuntimeError
- 30-second timeout on the HTTP request

7. Issue Matching & Report Statuses

match_issues() iterates WATCHED_ISSUES in order. For each slot it finds the first node where title matches (case-insensitive) AND assignee.name matches (case-insensitive).

Description / notes extraction

Linear issue descriptions often contain markdown checklists (- [] / - [x]). _extract_notes() strips checklist lines and empty lines, joining remaining text as the note shown in Done rows.

Email summary header counts

Done	Issues with state.type == completed
Pending	Issues exist but not completed
Missing	No issue matched the slot
Total	Always 4 (one per watched slot)

8. HTML Email Report

report.py builds a self-contained HTML email (inline CSS, no external stylesheets except email-client-safe patterns). Layout:

- Dark header with date, team label, generation time
- Summary stat cards: Done / Pending / Missing / Total
- Progress bar (done/total percentage)
- Marco Durante section -- 2 issue rows
- Amer Roufail section -- 2 issue rows
- Footer with timestamp and link to linear.app/platformeq

Subject line format: [Jun 12, 2026] Sweep Report

Note: Special characters in user notes are embedded in HTML without escaping in v1. Trusted internal content only.

9. Gmail Delivery

Email is sent via Gmail SMTP (smtp.gmail.com, port 465, SMTP_SSL). The sending account uses a Gmail App Password (not the regular account password). App passwords require 2FA on the Google account.

From	gmail_address (secret)
To	send_to (secret)
Format	multipart/alternative with text/html part

Tip: Store the app password only in Secret Manager. Rotate by adding a new secret version.

10. Configuration & Secrets

Non-secret environment variables

TOOLS_GCP_PROJECT peq-tools (default)
TOOLS_DAILY_SWEEP_REPORT_S peq-tools-daily-sweep-report-config
TOOLS_DAILY_SWEEP_REPORT_D 1 = build report, no send
TOOLS_DAILY_SWEEP_REPORT_S 1 = skip SM + Linear + email (tests)

RY_RUN

Secret JSON schema (values in Secret Manager only)

```
{
  "gmail_address": "reports-bot@gmail.com",
  "gmail_app_password": "xxxx xxxx xxxx xxxx",
  "send_to": "team@platformeq.com",
  "linear_api_key": "lin_api_..."
}
```

Terraform creates the secret container in `infra/terraform/jobs.tf`. Values are added with `gcloud secrets versions add --` never committed to git or Terraform state.

```
gcloud secrets versions add peq-tools-daily-sweep-report-config --data-file=- <<'EOF'
{"gmail_address":"...", "gmail_app_password":"...", "send_to":"...", "linear_api_key":"..."}
EOF
```

11. GCP Integration (Detailed)

The job runs in GCP project peq-tools (region us-central1). It uses the same platform primitives as the rest of tools-gcp: Secret Manager, Artifact Registry, Cloud Run service account identity, and Cloud Logging.

GCP services involved

Secret Manager	Stores Linear + Gmail credentials (JSON)
Artifact Registry	Docker image: .../tools-gcp/tools-gcp:latest
Cloud Run (Job)	Target host for scheduled execution (planned)
Cloud Scheduler	Cron trigger Mon-Fri 9am ET (planned)
Cloud Logging	Stdout/stderr from job runs
Cloud Monitoring	Optional alerts on job failure
IAM	Service account permissions, no key files

Terraform resources (jobs.tf)

- google_secret_manager_secret.daily_sweep_report_config
- IAM: tools-gcp-run SA gets secretAccessor on job secret
- IAM: developers get secretAccessor for local dev
- Output: daily_sweep_report_secret

How the job reads secrets at runtime

```

Cloud Run Job container starts
|
| Attached identity: tools-gcp-run@peq-tools.iam.gserviceaccount.com
| (no JSON key file on disk)
v
tools-daily-sweep-report runs
|
v
tools.secrets.get_secret("peq-tools-daily-sweep-report-config")
|
v
Secret Manager API (ADC / metadata server)
|
v
Returns latest secret version payload (JSON)
|
v
Parsed into ReportSecrets dataclass

```

Locally, ADC comes from gcloud auth application-default login. On Cloud Run, the metadata server provides tokens for the attached service account automatically.

12. Hosting on GCP: Current vs Target

Current state (implemented today)

- Job code complete under src/jobs/daily-sweep-report/
- CLI: tools-daily-sweep-report (also make daily-sweep-report for dry-run)
- Secret container defined in Terraform (jobs.tf)
- Unit tests in tests/jobs/daily_sweep_report/
- Same Docker image as the FastAPI service (tools-gcp)
- Manual or local execution; Cloud Scheduler not yet in Terraform

Target state (recommended production hosting)

Batch jobs like this should run as a Cloud Run Job, not the long-lived Cloud Run Service (which runs tools-serve / FastAPI). Cloud Run Jobs start, run the command, and exit -- ideal for cron workloads.

Cloud Run Service	tools-serve -- /health, /ready (always on)
Cloud Run Job	tools-daily-sweep-report -- runs and exits
Image	Same image; different command override

Note: Cloud Run Job + Scheduler Terraform is the next infrastructure step. The job code is ready for it today.

13. Cloud Run Job + Scheduler Architecture

```

Cloud Scheduler (cron: Mon-Fri 14:00 UTC ~= 9am EST)
|
| HTTP POST + OIDC token
v
Cloud Run Jobs API
|
v
Cloud Run Job: daily-sweep-report
| image: us-central1-docker.pkg.dev/peq-tools/tools-gcp/tools-gcp:latest
| command: ["tools-daily-sweep-report"]
| serviceAccount: tools-gcp-run@peq-tools.iam.gserviceaccount.com
| env: TOOLS_GCP_PROJECT=peq-tools
|      TOOLS_DAILY_SWEEP_REPORT_SECRET_NAME=peq-tools-daily-sweep-report-config
v
Container execution (~30-60 seconds)
|-- Secret Manager (read credentials)
|-- Linear API (fetch issues)
|-- Gmail SMTP (send email)
v
Exit 0 (success) or non-zero (failure logged)

```

Suggested schedule

```

# Cloud Scheduler (America/New_York aware)
# Option A: 0 14 * * 1-5 (14:00 UTC -- 9am EST standard time; 10am EDT)
# Option B: timezone=America/New_York cron=0 9 * * 1-5 (always 9am local)

```

Option B is preferred so the report always runs at 9:00 AM Eastern regardless of DST. The original GitHub workflow used `0 14 * * 1-5 UTC`.

Deployment pipeline

GitHub Actions (WIF) builds and pushes the Docker image on merge to main. The Cloud Run Job references `:latest` or a pinned digest. Updating the job does not require redeploying Scheduler -- only the Job template image/command.

14. IAM, Identity & Security Model

Principles (from docs/SECRETS.md)

- No service account JSON keys on disk or in git
- Secret values only in Secret Manager
- Secret names in code and Terraform are fine
- Audit logging on Secret Manager DATA_READ enabled

Identity matrix

Local dev	Your user ADC via gcloud auth application-default login
Cloud Run Job	tools-gcp-run@peq-tools.iam.gserviceaccount.com
Scheduler invoker	Dedicated SA with run.invoker on the Job (planned)
CI (GitHub Actions)	TOOLS_SKIP_GCP=1 -- no GCP in unit tests

Minimum IAM for the runtime SA

- roles/secretmanager.secretAccessor on peq-tools-daily-sweep-report-config
- roles/logging.logWriter (project) -- Cloud Run logs

The job does not need GCS, BigQuery, or Artifact Registry at runtime -- only Secret Manager read and outbound HTTPS (Linear + Gmail).

15. Local Development & Testing

```
# Install
make install

# Dry run (no GCP, no email)
make daily-sweep-report
# or: TOOLS_SKIP_GCP=1 tools-daily-sweep-report --dry-run

# Live run against real Linear + Gmail
gcloud auth application-default login
gcloud config set project peq-tools
tools-daily-sweep-report

# Tests
make test # includes 8 daily-sweep-report tests
```

Test coverage

- Monday -> Friday date logic
- Tuesday -> Monday date logic
- Issue matching by title + assignee
- Email subject and HTML content
- Secret schema validation
- run() with TOOLS_SKIP_GCP=1

16. Migration from GitHub Actions

Before (GitHub Actions)

- Cron in .github/workflows with inline Python heredoc
- Secrets: GMAIL_ADDRESS, APP_PASSWORD, SEND_TO, LINEAR_API_KEY
- Ran on ubuntu-latest GitHub runner

After (tools-gcp on GCP)

- Modular Python package under src/jobs/daily-sweep-report/
- Secrets in peq-tools-daily-sweep-report-config (JSON)
- Runs on Cloud Run Job in peq-tools (planned schedule)
- CI runs unit tests only; no secrets in GitHub

GitHub Actions can remain for deploy (WIF -> Artifact Registry -> Cloud Run). The report cron moves off GitHub entirely.

17. Operations, Monitoring & Troubleshooting

Where to look when something fails

Job logs	Cloud Logging -> Cloud Run Job execution logs
Secret access	Cloud Audit Logs -> Secret Manager DATA_READ
Scheduler	Cloud Scheduler execution history

Common failures

403 Secret Manager	SA missing secretAccessor on job secret
Linear 401/403	Invalid or expired linear_api_key in secret
Gmail auth error	Wrong app password or 2FA not enabled
Empty report / all Missing	Wrong date window or titles don't match
ModuleNotFoundError	pip install -e . after adding job package

Manual re-run

Execute the Cloud Run Job manually from Console or: `gcloud run jobs execute daily-sweep-report --region us-central1`

18. Future Jobs in tools-gcp

daily-sweep-report establishes the pattern for all future utilities:

- src/jobs/<kebab-name>/ -- isolated code
- tools-<kebab-name> CLI entry point
- Optional Secret Manager container in infra/terraform/jobs.tf
- Optional Cloud Run Job + Scheduler in Terraform
- Tests under tests/jobs/<snake_name>/

The FastAPI service (tools-serve) remains the platform health shell. Jobs share the Docker image but use different commands. This keeps one repo, one deploy pipeline, and many isolated batch utilities.